

Semantic Ontology

Platform Version: 0.0.4

Author: Meow Li

Copyright: Copyright (c) 2026 Meow Li. All Rights Reserved.

This document defines the system-level bridge ontology for Partner. It is the formal vocabulary that gadgets, rules, explanations, alert analysis, UI, tests, and future LLM-assisted gadget generation should share.

The ontology is needed because many bridge calls cannot be interpreted by auction-pattern enumeration alone. For example, `4N` may be quantitative, RKCB, old-style Blackwood, natural, or competitive/takeout depending on semantic context. A portable gadget must be able to ask the engine for formal bridge state such as agreed trump, pending transfer, forcing status, competitive pressure, or notrump focus instead of relying on private strings.

The current Python prototype has a simple `SemanticFact` trace. That trace is a temporary implementation detail. The target design is an ontology-backed semantic state with explicit merge, query, conflict, and explanation behavior.

The ontology also supplies the vocabulary and type system for the future Bridge System Language. BSL keywords should compile into ontology-backed IR effects and queries, not into arbitrary private strings.

1. Design Principles

1.1 Pattern Matching Is Local

`auction\_pattern` identifies the visible auction shape needed by a rule.

Pattern matching should answer:

- ? Did this local auction shape happen?
- ? Whose turn is it?
- ? Which call is being interpreted or selected?

Pattern matching should not be forced to encode every possible path to a bridge concept. A gadget should not need to enumerate every auction where `4N` is RKCB.

1.2 Semantic State Is Derived

The client submits visible bridge information:

- ? system,
- ? auction,
- ? hand,
- ? dealer,
- ? vulnerability,
- ? scoring,
- ? seat.

The engine derives semantic state by replaying the auction through active gadgets. Clients should not submit hidden state such as "heart transfer pending" or "hearts agreed".

1.3 Ontology Terms Are Executable

Every formal ontology term must define expected engine behavior:

- ? how a rule writes it,
- ? how another rule queries it,
- ? how conflicts are detected,
- ? how it appears in internal origin,
- ? how public meaning can be generated from it when appropriate.

Natural-language labels are display text. They are not the source of truth.

1.4 Public Meaning And Internal State Are Different

Public meaning is partnership disclosure.

Internal semantic state is for execution, training, debugging, and later AI/LLM workflows.

Example: a rule may publicly explain `2D` as "transfer to hearts". Internally, the same call may set a formal transfer object, mark responder as the owner of the shown heart length, and create an obligation for opener to accept, superaccept, or handle interference.

1.5 Gadgets Share One Vocabulary

Portable gadgets must not invent private semantic strings that other gadgets cannot understand. If a gadget needs a new concept, the concept should be added to this ontology or namespaced as an explicit extension with documented executable behavior.

2. Core Semantic State Object

The target engine should derive a `SemanticState` while replaying the auction.

Expected top-level domains:

```
semantic_state:
  auction_context: {}
  roles: {}
  shown_hands: {}
  call_history: []
  call_nature: []
  suit_state: {}
  notrump_state: {}
  transfer_state: []
  relay_state: []
  forcing_state: {}
  obligations: []
  competitive_state: {}
  slam_state: {}
  alert_state: []
  diagnostics: []
```

The exact Python class names can evolve, but the domains above should remain conceptually stable.

3. Auction Context

Auction context terms describe where the auction is, independent of hand meaning.

``seat_number``

Values: ``1``, ``2``, ``3``, ``4``.

Executable behavior:

- ? Derived from the number of initial passes before the first non-pass call.
- ? Can be used by rules that differ by opening position, passed-hand status, or Drury-style auctions.
- ? It is a representation of the same visible auction prefix, not a separate concept from ``P``, ``PP``, or ``PPP``.

Examples:

- ? ``1N`` has ``seat_number: 1``.
- ? ``P1N`` has ``seat_number: 2``.
- ? ``PP1H`` has ``seat_number: 3``.
- ? ``PPP1S`` has ``seat_number: 4``.

``auction_phase``

Recommended values:

- ? ``opening``
- ? ``response``
- ? ``opener_rebid``
- ? ``responder_rebid``
- ? ``competitive``
- ? ``balancing``
- ? ``slam``
- ? ``passed_hand``
- ? ``forced_action``

Executable behavior:

- ? Derived from auction history and semantic state.
- ? Rules may require a phase, but should not use phase as a substitute for more precise concepts when precision is needed.
- ? Multiple phase tags may be true at once. For example, an auction can be both ``competitive`` and ``slam``.

``auction_status``

Recommended values:

? `live`

? `complete`

? `illegal`

? `undefined`

Executable behavior:

? The engine should eventually validate auction legality before rule selection.

? Undefined means legal or possibly legal, but no loaded rule describes it.

4. Side, Seat, And Role Ontology

The raw input uses absolute seats `n e s w`. Analysis often needs relative concepts.

`absolute\_seat`

Values: `n`, `e`, `s`, `w`.

Executable behavior:

? Always derived from dealer plus auction index.

? Used for table state and display.

`side`

Values:

? `ours`

? `theirs`

Executable behavior:

? Derived from the acting seat relative to the requested user's side.

? Used for internal analysis, candidate generation, and explanations.

`partnership`

Values:

? `ns`

? `ew`

Executable behavior:

? Derived from absolute seat.

? Used with vulnerability and opponent/partner relationships.

`role`

Recommended values:

? `opener`

? `responder`

? `overcaller`

? `advancer`

? `intervenor`

? `doubler`

? `redoubler`

? `balancer`

? `captain`

? `describer`

Executable behavior:

? Roles are derived from the auction and can change or accumulate.

? A rule may assign a role to a seat after a call.

? Role queries must specify whose role is being queried when ambiguity is possible.

Example:

```
effects:  
  - assign_role:  
      subject: actor  
      role: opener
```

5. Vulnerability And Scoring

`vulnerability\_state`

Raw values: `none`, `ns`, `ew`, `both`.

Executable behavior:

? Stored from board environment.

? Used to derive relative vulnerability.

`vulnerability\_relation`

Values:

- ? `favorable`
- ? `unfavorable`
- ? `equal\_none`
- ? `equal\_both`

Executable behavior:

- ? Derived from `vulnerability\_state` and partnership.
- ? Used by rule algorithms, not by auction-pattern matching.

`scoring`

Recommended values:

- ? `MP`
- ? `IMP`
- ? `rubber`
- ? `practice`

Executable behavior:

- ? Environment input.
- ? Can affect judgmental selection algorithms, risk, game tries, and preempt style.

6. Call Nature

Call nature describes what a call is as a bridge action.

Recommended values:

- ? `natural`
- ? `artificial`
- ? `conventional`
- ? `relay`
- ? `puppet`
- ? `transfer`
- ? `preemptive`
- ? `constructive`
- ? `competitive`
- ? `penalty`
- ? `takeout`
- ? `responsive`
- ? `negative\_double`

? `support\_double`
?
? `redouble`
?
? `pass`
?
? `signoff`
?
? `invite`
?
? `game\_force`
?
? `slam\_try`
?
? `keycard\_ask`
?
? `blackwood\_ask`
?
? `quantitative\_invite`
?
? `control\_bid`

Executable behavior:

? Public meaning may display call nature.
?
? Alert analysis may query call nature.
?
? Later rules may require call nature, but should prefer precise semantic terms when needed.

Example:

```
meaning:  
  call_nature: artificial  
  action_type: transfer  
  target_suit: H
```

7. Shown Hand Attributes

Shown hand attributes describe what the auction says about a player.

`hcp\_range`

Fields:

? `min`
?
? `max`
?
? `source`
?
? `confidence`

Executable behavior:

? Multiple rules can narrow a range.
?
? Conflicting ranges should generate diagnostics.
?
? The state must preserve origin rules for training/debug output.

Example:

```
shows:
- subject: actor
  hcp:
    min: 15
    max: 17
```

`suit\_length`

Fields:

```
? `suit`: `C D H S`
? `min`
? `max`
? `exact`
? `subject`
```

Executable behavior:

- ? Used for later selection and explanation.
- ? Multiple facts should merge by tightening ranges when possible.
- ? Impossible intersections create conflict diagnostics.

Example:

```
shows:
- subject: actor
  suit: H
  length:
    min: 5
```

`shape\_class`

Recommended values:

```
? `balanced`
? `semibalanced`
? `unbalanced`
? `single_suiter`
? `two_suiter`
? `three_suiter`
? `minor_two_suiter`
? `major_two_suiter`
```

Executable behavior:

- ? Used as a broad descriptor.
- ? Should not replace explicit suit-length constraints when exact behavior matters.

`stopper`

Fields:

? `suit`

? `status`: `shown`, `denied`, `asked`, `unknown`

? `quality`: optional structured metric

Executable behavior:

? Used by notrump decisions, competitive auctions, and Lebensohl-style structures.

`control`

Fields:

? `suit`

? `round`: `first`, `second`, `first\_or\_second`

? `status`: `shown`, `denied`, `asked`

Executable behavior:

? Used in slam auctions and control-bidding gadgets.

8. Suit, Trump, And Notrump State

This project uses the term **suit** for `C D H S N`. For bridge trump, only `C D H S` can become a trump suit; `N` means notrump.

`proposed\_suit`

Meaning: a suit suggested by a call but not necessarily agreed.

Executable behavior:

? Can be overwritten or accumulated.

? Does not by itself make later `4N` RKCB.

`target\_suit`

Meaning: the intended suit in a transfer, relay, or puppet action.

Executable behavior:

? Used by transfer and puppet rules.

? May become proposed or agreed later depending on the convention.

`agreed\_suit`

Meaning: partnership has agreed a trump suit.

Executable behavior:

- ? Can be set explicitly by raise, fit-showing call, Texas transfer agreement, or another convention-specific effect.
- ? Later slam gadgets may query it.
- ? A call that merely completes Jacoby transfer does not automatically agree trump unless the gadget rule says so.

`trump\_candidate`

Meaning: likely future trump suit but not fully agreed.

Executable behavior:

- ? Useful for auctions where a suit is strongly implied but still negotiable.
- ? RKC rules should normally require ``agreed_suit``, not only ``trump_candidate``, unless a gadget explicitly allows that style.

`notrump\_focus`

Meaning: the auction is oriented around notrump.

Executable behavior:

- ? Supports quantitative 4N, stopper asks, and notrump game/slam decisions.
- ? Conflicts with an agreed trump suit only if a rule declares them mutually exclusive.

Example:

```
effects:
- set_notrump_focus:
  subject: partnership
  status: active
```

9. Transfer State

Transfers need formal state because later calls may accept, superaccept, decline, retransfer, or define special behavior after interference.

`transfer`

Fields:

- ? ``owner``: player who made the transfer call.
- ? ``acceptor``: partner expected to act.
- ? ``bid_suit``: suit actually bid.
- ? ``target_suit``: intended suit.

? `status`: `pending`, `accepted`, `superaccepted`, `declined`, `retransfer\_requested`, `completed`
? `interference`: optional structured object
? `origin`: rule origin

Executable behavior:

? A transfer-creating rule writes `status: pending`.
? Completion changes status to `accepted` or `completed`.
? Superaccept changes status to `superaccepted`.
? Interference can modify obligations without destroying the transfer.
? Later rules query the transfer object by target suit, owner, acceptor, and status.

Example transfer creation:

```
effects:
- create_transfer:
  owner: actor
  acceptor: partner
  bid_suit: D
  target_suit: H
  status: pending
```

Example normal completion:

```
requires:
  transfer:
    target_suit: H
    status: pending
effects:
- update_transfer:
  target_suit: H
  status: completed
- set_proposed_suit:
  suit: H
  source: transfer_completion
```

Example doubled transfer retransfer:

```
context:
  auction_pattern: "1NP2DXP"
call: R
requires:
  transfer:
    target_suit: H
    status: pending
  interference:
    over_call: X
    over: transfer_bid
effects:
- update_transfer:
  target_suit: H
  status: retransfer_requested
```

The exact auction above is an example only. The important system rule is that the retransfer rule should query formal transfer and interference state.

10. Relay And Puppet State

`relay`

Fields:

? `asker`

? `describer`

? `topic`

? `status`: `pending`, `answered`, `broken`

Executable behavior:

? Relay systems create obligations for the describer.

? The reply consumes or advances relay state.

`puppet`

Fields:

? `asker`

? `describer`

? `requested\_suit\_or\_shape`

? `status`

Executable behavior:

? Similar to relay, but specifically asks partner to bid a suit or describe holding in a controlled way.

11. Forcing And Obligation State

`forcing\_status`

Recommended values:

? `not\_forcing`

? `forcing\_one\_round`

? `game\_forcing`

? `slam\_forcing`

? `pass\_forcing`

Executable behavior:

? A rule can set forcing status for a partnership or a seat.

? Default policy should consult forcing status before choosing pass.

? Conflicting forcing statuses should merge toward the strongest active force unless a rule explicitly ends the force.

`obligation`

Fields:

? `subject`: seat or role expected to act.

? `action\_type`: formal expected action.

? `target`: optional suit, level, call type, or semantic object.

? `status`: `active`, `satisfied`, `cancelled`, `failed`

? `source`

Executable behavior:

? Transfer acceptance, relay replies, forced bids, and forcing pass actions should be obligations.

? A selected or historical call can satisfy an obligation.

? Unsatisfied obligations should appear in diagnostics.

Example:

```
effects:
- create_obligation:
  subject: partner
  action_type: accept_transfer_or_superaccept
  target_suit: H
  status: active
```

12. Competitive State

Competitive auctions need formal representation of opponent action.

`interference`

Fields:

? `actor\_side`: `theirs`

? `call`

? `over`: semantic object or call index

? `level`

? `suit`

? `type`: `bid`, `double`, `redouble`, `preempt`, `cuebid`

Executable behavior:

? Created when opponents bid, double, or redouble over an agreement sequence.

? Used to decide whether systems are on, off, modified, or replaced by competitive agreements.

`double\_meaning`

Recommended values:

- ? *`penalty`*
- ? *`takeout`*
- ? *`negative`*
- ? *`responsive`*
- ? *`support`*
- ? *`lead\_directing`*
- ? *`card\_showing`*
- ? *`cooperative`*
- ? *`maximal`*

Executable behavior:

- ? A double rule must define which meaning applies under its conditions.
- ? If multiple double meanings match, the engine should report ambiguity unless a resolver chooses one.

`systems\_status`

Values:

- ? *`on`*
- ? *`off`*
- ? *`modified`*

Executable behavior:

- ? Applies to a named gadget or convention family.
- ? Interference rules can change status.

Example:

```
effects:
- set_systems_status:
    gadget_id: four_way_jacoby_transfer
    status: modified
    reason: transfer_bid_doubled
```

13. Slam And Keycard State

`slam\_interest`

Values:

- ? *`none`*

? `invite`

? `try`

? `force`

Executable behavior:

? Created by quantitative tries, control bids, keycard asks, cue bids, or explicit slam tries.

? Can be queried by slam gadgets.

`keycard\_context`

Fields:

? `trump\_suit`

? `asker`

? `responder`

? `method`: `0314`, `1430`, or gadget-defined

? `status`: `pending`, `answered`

Executable behavior:

? A keycard ask requires a known trump source unless the gadget explicitly defines another rule.

? Answers satisfy the pending keycard context.

`four\_notrump\_resolution`

`4N` is not a meaning by itself. It is a call that must be resolved against semantic state.

Recommended candidate meanings:

? `quantitative\_invite`

? `keycard\_ask`

? `blackwood\_ask`

? `natural`

? `competitive\_takeout`

? `two\_suited\_takeout`

Executable behavior:

1. Gather all active `4N` meaning candidates from active gadgets.
2. Evaluate each candidate's `requires` block against semantic state.
3. Prefer candidates with the most specific satisfied requirements only when the resolver declares a deterministic rule.
4. If multiple candidates remain equally valid, emit an ambiguity diagnostic.
5. If no candidate matches, the auction is undefined for `4N`.

Example RKCB candidate:

```
call: 4N
requires:
  agreed_suit:
    exists: true
  slam_interest:
    min: try
meaning:
  call_nature: conventional
  action_type: keycard_ask
effects:
  - create_keycard_context:
      trump_suit: agreed_suit
      method: 1430
      status: pending
```

Example quantitative candidate:

```
call: 4N
requires:
  notrump_focus:
    status: active
  agreed_suit:
    exists: false
meaning:
  call_nature: natural
  action_type: quantitative_invite
effects:
  - set_slam_interest:
      level: invite
      suit: N
```

14. Alert And Disclosure State

`alertability`

Fields:

? `jurisdiction`: for example `ACBL`

? `rules\_version`: optional date/version

? `status`: `alertable`, `not\_alertable`, `announcement`, `delayed\_alert`, `unknown`

? `source`: `hard\_coded`, `rules\_engine`, `manual\_override`

Executable behavior:

? Near-term rules may hard-code `meaning.alertable`.

? Long-term alert analysis should evaluate current rules for the jurisdiction and version.

? Alertability belongs in public/disclosure output, with rule origin.

`public\_disclosure`

Fields:

? `summary`

? `details`

? `call\_nature`

? `shown\_values`

? `alertability`

Executable behavior:

? Generated from `meaning` plus ontology state.

? Must not expose private candidate comparisons unless the product explicitly shows a training/debug layer.

15. Rule Schema Direction

The target rule model should distinguish these sections:

```
rules:
- id: example_rule
  context: {}
  call: 2D
  applies_when: {}
  requires: {}
  selection: {}
  meaning: {}
  shows: []
  effects: []
  clears: []
```

Expected behavior:

? `context` matches visible auction shape.

? `call` is the selected or interpreted call.

? `applies\_when` evaluates the actor's actual hand and environment for bid selection.

? `requires` queries semantic state.

? `selection` chooses among candidate calls.

? `meaning` defines public partnership explanation.

? `shows` records what the call says about a hand.

? `effects` updates semantic state.

? `clears` removes or satisfies temporary semantic state such as pending obligations.

The existing prototype uses `selection`, `meaning`, and `semantic\_effects`. Future schema evolution should migrate toward the clearer structure above while preserving backward compatibility during transition.

16. Conflict And Ambiguity Rules

The engine must diagnose semantic problems instead of silently choosing an arbitrary meaning.

Required diagnostics:

? two matching rules give incompatible public meanings for the same call,

- ? two matching candidates produce tied selection scores without a resolver,
- ? shown hand ranges are impossible after merging,
- ? an obligation remains active when the auction moves past it,
- ? a call asks for a semantic object that does not exist,
- ? a call has multiple valid semantic interpretations, such as quantitative `4N` and RKCB `4N`.

Resolvers are allowed, but they must be explicit and documented.

17. Ontology Extension Policy

If a gadget needs a concept not listed here:

1. Prefer an existing ontology term if the meaning and executable behavior match.
2. If not, add a new formal term to this document.
3. If the concept is experimental or private, namespace it clearly and document its query/write behavior.
4. Do not rely on arbitrary prose or private strings as the only machine-readable state.

18. Implementation Milestone Target

The next semantic milestone is an operable 2/1 plus after-1N system where:

- ? the 2/1 gadget defines the `1N` opening and other basic openings,
- ? the four-way Jacoby transfer gadget defines after-1N transfer responses and continuations,
- ? transfers create formal transfer state,
- ? completions and superaccepts query and update transfer state,
- ? later routes such as Texas and Smolen can compete by formal hand/environment/semantic conditions,
- ? `4N` examples can be resolved by semantic context instead of auction enumeration.